SECTION 5: INPUT-OUTPUT

Lecture 17: Formatted PRINT Statements

PRINT and WRITE statements:

PRINT *format-spec, output-list*

Where the format-spec is one of

*

character constant i.e., formatting='(3F6.2)'

format statement number i.e., `100 FORMAT(3F6.2)`

For example

```
a=1.; b=2.; c=3.
PRINT*, a, b, c
PRINT '(3F10.4)', a, b, c
PRINT 100, a, b, c
100  FORMAT(3F10.4)
PRINT*,2./3.
```

The output from these PRINT statements looks like this

g95 output:

```
12345678901234567890123456789 0 (I added this line)
 1. 2. 3.
    1.0000    2.0000    3.0000
    1.0000    2.0000    3.0000
    0.6667
```

gfortran output:

```
12345678901234567890123456789 0
   1.00000000       2.00000000       3.00000000
    1.0000    2.0000    3.0000
    1.0000    2.0000    3.0000
    0.6667
```

`PRINT` is not often used any more, except for debugging help.

Next let's look at formatted `WRITE` statements which offer much greater control, but are more complex.

Lecture 18: Formatted WRITE Statements

WRITE *(control-list) output-list*

The control-list includes:

A unit specifier

A format specifier

An advance specifier

Examples:

Write(*,*) a,b,c "free" format

WRITE(UNIT=6,FMT=*) write to screen in free format

WRITE(6,100) write to screen using a 100 FORMAT statement

WRITE(6,100,ADVANCE='NO') same but do not advance to next line

Format Descriptors

I	rIw	Integer output (right justified)
F	rFw.d	Real output
E	rEw.d	Exponential notation for real output
ES	rESw.d	Scientific notation
L	rLw	Logical (output either T or F, right justified)
A	rA or rAw	Character (w is width of field)
X	nX	Horizontal positioning
T	Tc	Tab descriptor (moves to column c)
/		Go to next line

Example:

```
PROGRAM write_format
REAL :: a=1,b=2,c=3
CHARACTER(20) :: d,e,f
d='The value for a is: '
e='The value for b is: '
f='The value for c is: '
PRINT*; PRINT*
WRITE(*,*)a,b,c   !free format
PRINT*
WRITE(6,100)a,b,c
100 FORMAT(2F10.3,F6.0)
PRINT*
WRITE(*,'(3E10.3)')a,b,c
PRINT*
WRITE(*,'(3ES10.3)')a,b,c
PRINT*
write(*,200)a,b,c
200 FORMAT(5X,F3.1,5X,F3.1,5X,F3.1)
PRINT*
WRITE(*,300)a,b,c
300 FORMAT(T5,f5.1,T10,F5.1,T20,F5.1)
PRINT*
WRITE(*,400)a,b,c
400  FORMAT(F3.1,/,F3.1,/,F3.1)
PRINT*
WRITE(*,500)a,b,c
500 FORMAT(f5.4,2X,f5.3,2X,f5.2)
```

```
PRINT*
WRITE(*,600)a,b,c
600 FORMAT('The value for a is:',F4.1,/, &
           'The value for b is:',F4.1,/, &
           'The value for c is:',F4.1)
PRINT*
WRITE(*,'(A)')d,e,f
PRINT*
WRITE(*,700)d,a,e,b,f,c
700  FORMAT(A,f5.2,3X,A,f5.2,/,A,f5.2)
END PROGRAM write_format
```

```
G:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES>a.exe

 1. 2. 3.
     1.000     2.000    3.
 0.100E+01 0.200E+01 0.300E+01
 1.000E+00 2.000E+00 3.000E+00
     1.0     2.0     3.0
      1.0  2.0       3.0
1.0
2.0
3.0
*****  2.000   3.00
The value for a is: 1.0
The value for b is: 2.0
The value for c is: 3.0

The value for a is:
The value for b is:
The value for c is:

The value for a is:  1.00   The value for b is:  2.00
The value for c is:  3.00
```

NOTE: You may see a 1X in older Fortran code FORMAT statements as the first entry in the statement. This first column was a control character used to direct

output to a printer to control page layout (i.e., blank: single spacing; 0: double spacing; 1: skip to new page; +: no spacing). Should not need to worry about this with new codes.

Lecture 19: Formatted Input and File Processing

File Processing

Opening files for reading and writing:

Minimum that you usually need (unit and file name):

OPEN(UNIT=xx, FILE='name') !opens a file to read from

WRITE(UNIT=xx, FILE='name') !write to a file associated with unit=xx on disk

Example OPEN and WRITE useage:

```
OPEN(10, file='output_data')
WRITE(10,*)a, b, c
```

In general:

OPEN(*open_list*)

Where commonly used *open_list* options include:

UNIT= *integer_expression*

FILE= *character_expression*

IOSTAT= *integer_variable* if open successful integer_variable returns a zero; positive number if not successful

STATUS= *character_expression* 'OLD', 'NEW', 'REPLACE', 'SCRATCH', 'UNKNOWN'

ACTION= *character_expression* 'READ', 'WRITE', or 'READWRITE'

By Example:

Input Files data1 and data2:

```
G:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES>more data1
   4.        5.          6.
12345678911234567891234567891
G:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES>more data2
   4    5              6
12345678911234567891234567891
G:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES>
```

```
PROGRAM formatted_input
IMPLICIT none
REAL :: a,b,c
INTEGER :: istatus

PRINT*
OPEN(UNIT=20,file="data1",status='old',iostat=istatus)
PRINT*,'istatus 20= ',istatus
OPEN(UNIT=21,file='data2',status='old',iostat=istatus)
PRINT*,'istatus 21= ',istatus
PRINT*

READ(20,*,IOSTAT=istatus)a,b,c  !reads data from unit 20
PRINT*,'unit 20 istatus=',istatus
WRITE(*,*)a,b,c
REWIND(20)       !rewinds unit 20
PRINT*

READ(20,100,IOSTAT=istatus)a,b,c
100 FORMAT(3f10.3)
PRINT*,'unit 20 istatus=',istatus
WRITE(*,*)a,b,c
REWIND(20)
PRINT*
```

```
READ(21,100,IOSTAT=istatus)a,b,c
PRINT*,'unit 21 istatus=',istatus
WRITE(*,*)a,b,c
PRINT*

!  INCORRECT FORMAT STATEMENT
READ(20,101,IOSTAT=istatus)a,b,c
101 FORMAT(2f10.3,I10)
PRINT*,'unit 20 istatus=',istatus
REWIND(20)
WRITE(*,*)a,b,c
PRINT*

CLOSE(20); CLOSE(21)
END PROGRAM formatted_input
```

```
E:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES>g95 formatted_input.f90

E:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES>a.exe

 istatus 20=  0
 istatus 21=  0

 unit 20 istatus= 0
 4. 5. 6.

 unit 20 istatus= 0
 4. 5. 6.

 unit 21 istatus= 0
 0.045 0. 0.006

 unit 20 istatus= 205
 4. 5. 0.006


E:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES>
```

Lecture 20: Unformatted Files

Formatted files contain recognizable characters that can be read by anyone editing the file. This causes some issues in terms of file sizes, and the work a processor needs to do to convert from the formatted representation to the processors internal representation.

Unformatted files overcome these disadvantages, but may be processor specific.

Unformatted files are particularly useful for **scientific computing applications** where data sets may be extremely large.

Let's look at an example

```
PROGRAM unformatted
IMPLICIT NONE
REAL, DIMENSION(1000,1000) :: a, b
INTEGER :: count_start, count_end, count_rate
REAL :: wall_time
OPEN(UNIT=10,file='formatted_data')
OPEN(UNIT=11,file='unformatted_data',FORM='unformatted')

CALL RANDOM_SEED()
CALL RANDOM_NUMBER(a)
PRINT*

CALL SYSTEM_CLOCK(count_start,count_rate)
WRITE(10,*)a
CALL SYSTEM_CLOCK(count_end)
wall_time=REAL(count_end-count_start)/REAL(count_rate)
PRINT*,'Formatted write wall time=',wall_time

CALL SYSTEM_CLOCK(count_start,count_rate)
WRITE(11)a
CALL SYSTEM_CLOCK(count_end)
wall_time=REAL(count_end-count_start)/REAL(count_rate)
PRINT*,'Unformatted write wall time=',wall_time
PRINT*

CLOSE(10)
CLOSE(11)
```

```
OPEN(UNIT=20,file='formatted_data')
OPEN(UNIT=21,file='unformatted_data',FORM='unformatted')

CALL SYSTEM_CLOCK(count_start,count_rate)
READ(20,*)a
CALL SYSTEM_CLOCK(count_end)
wall_time=REAL(count_end-count_start)/REAL(count_rate)
PRINT*,'Formatted read wall time=',wall_time

CALL SYSTEM_CLOCK(count_start,count_rate)
READ(21)b
CALL SYSTEM_CLOCK(count_end)
wall_time=REAL(count_end-count_start)/REAL(count_rate)
PRINT*,'Unformatted read wall time=',wall_time

PRINT*
END PROGRAM unformatted
```

OUTPUT:

```
E:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES>g95 -r8 -i8 unformatted.f90

E:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES>a.exe

 Formatted write wall time= 2.8064
 Unformatted write wall time= 0.425

 Formatted read wall time= 1.4416
 Unformatted read wall time= 0.0625


E:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES>dir *_data
 Volume in drive E has no label.
 Volume Serial Number is 024B-0197

 Directory of E:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES

06/18/2019  10:51 AM        19,270,196 formatted_data
06/18/2019  10:51 AM         8,000,008 unformatted_data
               2 File(s)     27,270,204 bytes
               0 Dir(s)  19,459,932,160 bytes free

E:\FORTRAN_COURSE\SECTION5-INPUT_OUTPUT\CODES>
```